CGE-P LAB NOTES

Lab 2.5: IaC as Compliance Evidence (AWS)

A reviewed, signed, immutably-stored Terraform commit is stronger evidence than a screenshot. This lab builds the vault that holds your evidence and the script that puts it there.

It also confronts a trap that is easy to walk into: capturing raw Terraform state into an immutable bucket. If that state contains a secret, you have just made the secret permanently undeletable. Step 3 deals with it.

For reading. Code lines wrap to fit the page; copy code from docs/02_05_iac_as_compliance_evidence.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/02_05_iac_as_compliance_evidence.md` in your clone, not from the PDF.

Learning objectives

- Define chain of custody in terms of integrity, attribution, and reproducibility.
- Build an S3 Object Lock vault that refuses deletion by design.
- Capture a workspace's evidence, hash it, bundle it, and upload it with a recorded VersionId.
- Recognize that evidence collection is itself a data-handling risk, and mitigate it before it becomes permanent.

Controls implemented

Control	Enforced by
AU-9, AU-9(3)	Object Lock retention; the vault refuses deletion
AU-11	Retention mode and duration, chosen explicitly
SC-8	Bucket policy denying non-TLS
SC-12, SC-13, SC-28	Vault CMK with rotation, SSE-KMS
AC-3, AC-6	Public access block, ACLs disabled
CP-9	Versioning, required by Object Lock
CM-6, CM-8	`default_tags`

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 2.2** state backend, and **Lab 2.3** completed with `evidence/lab-2-3/` populated and the workspace still on disk.
- AWS CLI v2, plus `jq` and either `sha256sum` or `shasum`.
- Terraform `>= 1.10`.
- Optional: Cosign, for the signing preview at the end. Lab 4.4 does it properly.

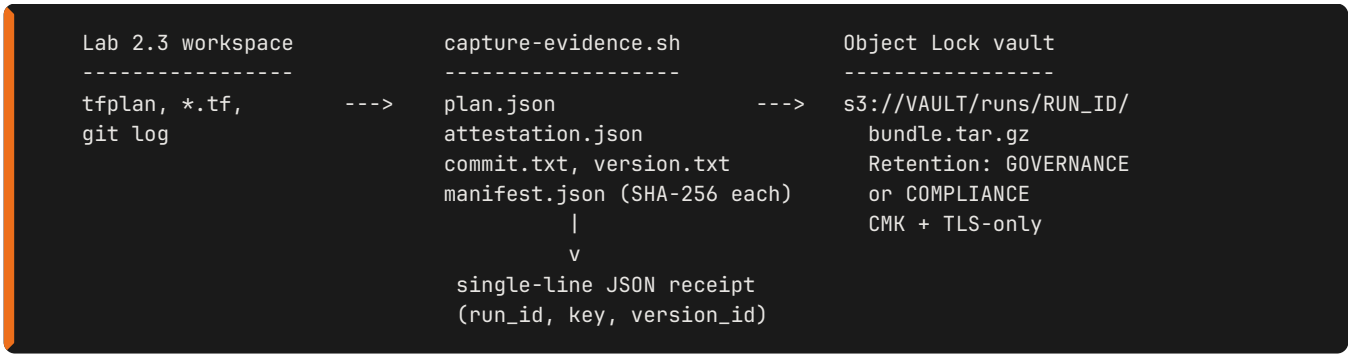
> **Read the Cleanup section before you apply this one.** The choice between `GOVERNANCE` and `COMPLIANCE` is not reversible and not symmetric: `GOVERNANCE` retention can be bypassed by a privileged caller, and `COMPLIANCE` cannot be bypassed by anyone, including the account root. Choose `COMPLIANCE` with a long retention and the bucket stays until every object expires, whatever you or AWS support would prefer.
>
> The defaults here are the safe ones. Change them on purpose, not on the way past.

Estimated time & cost

- Time: 45 to 60 minutes.
- Cost: Object Lock itself is free; you pay S3 storage on a few KB of bundles. **$1/month for the vault CMK.** Under $0.02 if you destroy same-day.

Choose `GOVERNANCE` mode with a 1-day retention for lab work. `COMPLIANCE` mode cannot be shortened or bypassed by anyone, including the account root, until retention expires. Pick `COMPLIANCE` with a long retention in a lab and you have created a bucket you will be paying for, and unable to empty, for the duration you chose. That is the correct behavior. It is also a genuinely expensive mistake.

Architecture



Note what is **not** in that list, and see Step 3: raw `terraform state pull` output.

Step-by-step walkthrough

Concept: why code is evidence

A screenshot of a console says "I once saw this." A Terraform plan committed to git, reviewed in a pull request, applied in CI, and stored in a vault that refuses deletion says "this is what was deployed, who reviewed it, when, and the artifact is unchanged since."

Auditors want three properties: integrity, attribution, reproducibility. Code-as-evidence delivers all three. Screenshots deliver none.

Step 1 Build the vault

Object Lock has one hard constraint: **it must be enabled at bucket creation.** There is no retrofit. If you get this wrong you destroy and recreate.

```
# terraform/primitives/evidence-vault/main.tf
terraform {
  required_version = ">= 1.10"

  backend "s3" {
    bucket       = "cgep-lab-tfstate-XXXXXXXX"
    key          = "labs/2-5/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    kms_key_id   = "arn:aws:kms:us-east-1:ACCOUNT:key/KEY-ID"
    use_lockfile = true
  }

  required_providers {
    aws    = { source = "hashicorp/aws", version = "~> 5.0" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project         = var.project_name
      Environment     = "evidence"
      ManagedBy       = "terraform"
      ComplianceScope = "cge-p-lab"
    }
  }
}

data "aws_caller_identity" "current" {}
data "aws_partition" "current" {}

resource "random_id" "suffix" {
  byte_length = 4
}

locals {
  account_id = data.aws_caller_identity.current.account_id
  partition  = data.aws_partition.current.partition
  vault_name = "${var.project_name}-grc-evidence-vault-${random_id.suffix.hex}"
}

# SC-12 / SC-13: the vault gets its own key, separate from the workload key
```

```
# in Lab 2.3. Separation of duties: whoever can read the data should not
# automatically be able to read the evidence about the data.
resource "aws_kms_key" "vault" {
  description             = "CMK for the GRC evidence vault"
  enable_key_rotation     = true
  deletion_window_in_days = var.kms_deletion_window_days
  policy                  = data.aws_iam_policy_document.kms.json
}

resource "aws_kms_alias" "vault" {
  name          = "alias/${var.project_name}-evidence-vault"
  target_key_id = aws_kms_key.vault.key_id
}

data "aws_iam_policy_document" "kms" {
  statement {
    sid       = "EnableAccountRootAdmin"
    effect    = "Allow"
    actions   = ["kms:*"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }
}

resource "aws_s3_bucket" "vault" {
  bucket              = local.vault_name
  object_lock_enabled = true # MUST be set at bucket creation. No retrofit exists.
}

resource "aws_s3_bucket_ownership_controls" "vault" {
  bucket = aws_s3_bucket.vault.id
  rule {
    object_ownership = "BucketOwnerEnforced"
  }
}

# Object Lock requires versioning. This is not optional.
resource "aws_s3_bucket_versioning" "vault" {
  bucket = aws_s3_bucket.vault.id
  versioning_configuration {
    status = "Enabled"
  }
}

# AU-9 / AU-11: the control that makes this a vault rather than a bucket.
resource "aws_s3_bucket_object_lock_configuration" "vault" {
  bucket     = aws_s3_bucket.vault.id
  depends_on = [aws_s3_bucket_versioning.vault]

  rule {
    default_retention {
      mode = var.lock_mode
      days = var.retention_days
```

```
    }
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "vault" {
  bucket = aws_s3_bucket.vault.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.vault.arn
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_public_access_block" "vault" {
  bucket                  = aws_s3_bucket.vault.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

data "aws_iam_policy_document" "vault" {
  # SC-8. Easily skipped on an internal bucket, and this is the one whose
  # contents are the audit record itself.
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.vault.arn, "${aws_s3_bucket.vault.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }

  # AU-9: nobody but the account root deletes this bucket. Object Lock protects
  # the objects; this protects the container.
  statement {
    sid       = "DenyBucketDeletion"
    effect    = "Deny"
    actions   = ["s3:DeleteBucket"]
    resources = [aws_s3_bucket.vault.arn]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "StringNotEquals"
      variable = "aws:PrincipalArn"
```

```
      values   = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }
}

resource "aws_s3_bucket_policy" "vault" {
  bucket = aws_s3_bucket.vault.id
  policy = data.aws_iam_policy_document.vault.json
}
```

```
# terraform/primitives/evidence-vault/variables.tf
variable "project_name" {
  type    = string
  default = "cgep-lab"
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,20}$", var.project_name))
    error_message = "project_name must be 3-21 lowercase alphanumerics or hyphens, starting with a
letter."
  }
}

variable "aws_region" {
  type    = string
  default = "us-east-1"
}

variable "lock_mode" {
  type        = string
  description = "GOVERNANCE for lab work; COMPLIANCE for real evidence."
  default     = "GOVERNANCE"
  validation {
    condition     = contains(["GOVERNANCE", "COMPLIANCE"], var.lock_mode)
    error_message = "lock_mode must be GOVERNANCE or COMPLIANCE."
  }
}

variable "retention_days" {
  type        = number
  description = "Default retention applied to every uploaded object."
  default     = 1

  validation {
    condition     = var.retention_days >= 1 && var.retention_days <= 3650
    error_message = "retention_days must be between 1 and 3650."
  }

  # COMPLIANCE mode is unbypassable by anyone including root, so a long
  # retention has to be a deliberate act rather than a default.
  validation {
    condition     = var.lock_mode != "COMPLIANCE" || var.retention_days <= 7 ||
var.acknowledge_compliance_mode
    error_message = "COMPLIANCE mode beyond 7 days requires acknowledge_compliance_mode = true. You
will not be able to delete these objects, ever, until retention expires."
  }
```

```
}

variable "acknowledge_compliance_mode" {
  type        = bool
  description = "Set true only if you intend evidence to outlive your ability to delete it."
  default     = false
}

variable "kms_deletion_window_days" {
  type    = number
  default = 7
  validation {
    condition     = var.kms_deletion_window_days >= 7 && var.kms_deletion_window_days <= 30
    error_message = "kms_deletion_window_days must be between 7 and 30."
  }
}
```

```
# terraform/primitives/evidence-vault/outputs.tf
output "vault_name" {
  value       = aws_s3_bucket.vault.id
  description = "Vault bucket name. Feed to capture-evidence.sh --vault."
}

output "vault_kms_key_arn" {
  value       = aws_kms_key.vault.arn
  description = "CMK protecting the vault."
}

output "lock_mode" {
  value       = var.lock_mode
  description = "Retention mode in force (AU-11 attestation)."
}

output "retention_days" {
  value       = var.retention_days
  description = "Default retention window in days (AU-11 attestation)."
}
```

That cross-variable validation on `retention_days` needs Terraform 1.9 or later. It exists because the most common way to hurt yourself in this lab is a `COMPLIANCE` vault with a 365-day retention created by accident.

GOVERNANCE vs COMPLIANCE. GOVERNANCE retention can be bypassed by a caller holding `s3:BypassGovernanceRetention` using `--bypass-governance-retention`. COMPLIANCE cannot be bypassed by anyone, including the account root, until the window expires. Use GOVERNANCE for labs so you can clean up. Use COMPLIANCE for real evidence, and mean it.

Step 2 Apply

```
export AWS_PROFILE=cgep
cd terraform/primitives/evidence-vault   # reference layout: cd reference/lab-2-5/terraform
terraform init
terraform fmt && terraform validate
terraform apply -auto-approve
VAULT=$(terraform output -raw vault_name)
```

Expected: `Apply complete! Resources: 10 added, 0 changed, 0 destroyed.`

Step 3 The problem with capturing state

The obvious capture script does this:

```
terraform state pull > "$BUNDLE_DIR/state.json"
```

then uploaded the bundle into an Object Lock vault.

Think about what that means. State holds every attribute of every resource in plaintext, including values marked `sensitive`: RDS passwords, IAM secret access keys, Lambda environment variables. Object Lock means the upload **cannot be deleted** until retention expires. In `COMPLIANCE` mode, not even by the account root.

So the happy path is: capture a secret, then make it permanently irretrievable-but-undeletable, in a bucket whose entire purpose is to be handed to an auditor.

Three ways out, and you should be able to defend your choice:

Approach	Trade-off
Capture the plan, not the state	`plan.json` describes intent and configuration, which is what policy engines and assessors actually read. Loses "what is currently deployed." **This is our default.**
Capture state, redacted	Keeps the deployed view. Redaction is a denylist, and denylists are wrong eventually.
Capture state, and never put secrets in Terraform	Correct in principle. Requires every secret to live in Secrets Manager with only ARNs in state. The right long-term answer, and not achievable by Chapter 2.

The script below takes the first, and provides `--include-state` for when you have consciously decided the workspace is secret-free. It refuses to run that flag against `COMPLIANCE` mode without an explicit override, because that combination is the irreversible one.

Step 4 Write `capture-evidence.sh`

```
#!/usr/bin/env bash
# scripts/capture-evidence.sh
# Usage:
#   capture-evidence.sh --workspace <path> --run-id <id> --vault <bucket>
#                       [--profile <p>] [--include-state] [--i-understand-compliance-mode]

set -euo pipefail

PROFILE_ARG=""
WORKSPACE=""
RUN_ID=""
VAULT=""
INCLUDE_STATE=0
ACK_COMPLIANCE=0

while [[ $# -gt 0 ]]; do
  case "$1" in
    --workspace) WORKSPACE="$2"; shift 2 ;;
    --run-id)    RUN_ID="$2";    shift 2 ;;
    --vault)     VAULT="$2";     shift 2 ;;
    --profile)   PROFILE_ARG="--profile $2"; shift 2 ;;
    --include-state) INCLUDE_STATE=1; shift ;;
    --i-understand-compliance-mode) ACK_COMPLIANCE=1; shift ;;
    *) echo "Unknown arg: $1" >&2; exit 2 ;;
  esac
done

[[ -z "$WORKSPACE" || -z "$RUN_ID" || -z "$VAULT" ]] && {
  echo "Usage: $0 --workspace <path> --run-id <id> --vault <bucket> [--profile <p>]" >&2
  exit 2
}

# Refuse the one irreversible combination: raw state into unbypassable storage.
if [[ $INCLUDE_STATE -eq 1 && $ACK_COMPLIANCE -eq 0 ]]; then
  MODE=$(aws $PROFILE_ARG s3api get-object-lock-configuration --bucket "$VAULT" \
           --query 'ObjectLockConfiguration.Rule.DefaultRetention.Mode' \
           --output text 2>/dev/null || echo "NONE")
  if [[ "$MODE" == "COMPLIANCE" ]]; then
    echo "REFUSING: --include-state against a COMPLIANCE-mode vault." >&2
    echo "Terraform state contains every attribute in plaintext, including secrets." >&2
    echo "COMPLIANCE retention cannot be bypassed by anyone, including root." >&2
    echo "Pass --i-understand-compliance-mode if this is genuinely what you want." >&2
    exit 3
  fi
fi

WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT

if command -v sha256sum >/dev/null 2>&1; then SHASUM="sha256sum"
elif command -v shasum   >/dev/null 2>&1; then SHASUM="shasum -a 256"
else echo "Need sha256sum or shasum" >&2; exit 2; fi
```

```
CAPTURED_AT=$(date -u +%Y-%m-%dT%H:%M:%SZ)
BUNDLE_DIR="$WORK/bundle-$RUN_ID"
mkdir -p "$BUNDLE_DIR"

# plan.json: the configuration and intent. The primary artifact.
( cd "$WORKSPACE" && [[ -f tfplan ]] \
    && terraform show -json tfplan > "$BUNDLE_DIR/plan.json" 2>/dev/null || true )

# The module's own attestation, if it emits one (Lab 2.3 does).
( cd "$WORKSPACE" && terraform output -json compliance_attestation \
    > "$BUNDLE_DIR/attestation.json" 2>/dev/null || true )

if [[ $INCLUDE_STATE -eq 1 ]]; then
  ( cd "$WORKSPACE" && terraform state pull > "$BUNDLE_DIR/state.json" 2>/dev/null || true )
fi

( cd "$WORKSPACE" && git log -1 --pretty=full > "$BUNDLE_DIR/commit.txt" 2>/dev/null \
    || echo "no git commit available" > "$BUNDLE_DIR/commit.txt" )
terraform version > "$BUNDLE_DIR/version.txt"

# manifest.json: filename, sha256, size, captured_at per file.
{
  echo "["
  FIRST=1
  for f in "$BUNDLE_DIR"/*; do
    base=$(basename "$f")
    [[ "$base" == "manifest.json" ]] && continue
    HASH=$($SHASUM "$f" | awk '{print $1}')
    SIZE=$(wc -c < "$f" | tr -d ' ')
    [[ $FIRST -eq 1 ]] && FIRST=0 || printf ","
    printf '\n  {"filename":"%s","sha256":"%s","size":%s,"captured_at_utc":"%s"}' \
      "$base" "$HASH" "$SIZE" "$CAPTURED_AT"
  done
  echo
  echo "]"
} > "$BUNDLE_DIR/manifest.json"

BUNDLE_TGZ="$WORK/bundle-$RUN_ID.tar.gz"
( cd "$WORK" && tar czf "$BUNDLE_TGZ" "bundle-$RUN_ID" )

KEY="runs/$RUN_ID/bundle.tar.gz"
VERSION_ID=$(aws $PROFILE_ARG s3api put-object \
  --bucket "$VAULT" --key "$KEY" --body "$BUNDLE_TGZ" \
  --query VersionId --output text)

printf '{"run_id":"%s","vault":"%s","key":"%s","version_id":"%s","captured_at_utc":"%s","includes_stat
e":%s}\n' \
  "$RUN_ID" "$VAULT" "$KEY" "$VERSION_ID" "$CAPTURED_AT" \
  "$([[ $INCLUDE_STATE -eq 1 ]] && echo true || echo false)"
```

Three deliberate details beyond the state guard. The bundle tarball is built inside the `mktemp` directory so the `trap` cleans it up, instead of being left in `/tmp`. The VersionId comes from `--query VersionId` rather than an `awk` parse of JSON. And the receipt records whether state was included, because that is a fact a future reader needs.

Step 5 Run it against Lab 2.3

```
chmod +x scripts/capture-evidence.sh

bash scripts/capture-evidence.sh \
  --workspace ../lab-2-3 \
  --run-id    test-001 \
  --vault     "$VAULT"
```

`../lab-2-3` is the workspace you applied in Lab 2.3. If you have built your own repository following the capstone layout, the same workspace lives at `terraform/primitives/compliant-s3`, and you pass that path instead. The script cares about finding a Terraform workspace, not about where you keep it.

Receipt:

```
{"run_id":"test-001","vault":"cgep-lab-grc-evidence-vault-XXXXXXXX","key":"runs/test-001/
bundle.tar.gz","version_id":"<base64-version-id>","captured_at_utc":"<iso-
utc>","includes_state":false}
```

The VersionId is the durable handle. Anything that points at this evidence later, including your OSCAL component's evidence URI in Lab 6.1, uses `s3://VAULT/KEY?versionId=...`. Without the version, you are pointing at "whatever is at this key now," which in a versioned bucket is not the same claim.

Step 6 Verify retention applied

```
aws s3api get-object-retention --bucket "$VAULT" \
  --key runs/test-001/bundle.tar.gz
```

```
{ "Retention": { "Mode": "GOVERNANCE", "RetainUntilDate": "<retain-until-utc>" } }
```

You did not set that explicitly. The bucket's default rule applied it at upload.

Step 7 The destructive test

This is the lesson.

The receipt printed a `version_id`. Read it back rather than transcribing it, for the same reason you read `backend.hcl` out of Lab 2.2's outputs:

```
VERSION_ID=$(aws s3api list-object-versions --bucket "$VAULT" \
  --prefix runs/test-001/ --query 'Versions[0].VersionId' --output text)
echo "$VERSION_ID"

aws s3api delete-object --bucket "$VAULT" \
  --key runs/test-001/bundle.tar.gz \
  --version-id "$VERSION_ID"
```

```
An error occurred (AccessDenied) when calling the DeleteObject operation:
Access Denied because object protected by object lock.
```

That rejection is the proof. The lesson is not that S3 has a feature called Object Lock. The lesson is that your evidence now resists silent tampering by an administrator who would prefer it not exist, and that resistance is a property of the storage rather than a promise in a policy document.

Try one more thing, and notice the difference:

```
aws s3api delete-object --bucket "$VAULT" --key runs/test-001/bundle.tar.gz
```

That one **succeeds**, because without `--version-id` it writes a delete marker rather than removing the object. The object is still there, still locked, still retrievable by version. Delete markers are how a versioned bucket says "hidden," not "gone." An auditor who does not know this will believe evidence was destroyed; an engineer who does knows where to look.

Step 8 Optional preview: sign the bundle

Read this before running it. Most people should skip this step.

Object Lock makes the bundle immutable. It does not prove who made it or when. That is what signing adds, and it is the whole subject of Lab 4.4.

The command below signs as **you**, and that has a consequence you cannot take back:

```
cosign sign-blob --yes --bundle bundle.sig.bundle /tmp/bundle-test-001.tar.gz
```

Cosign opens a browser, you log in with GitHub, Google or Microsoft, and Fulcio issues a certificate naming **the email address of the account you used**. That certificate and its signature are written to Rekor, a **public transparency log that is append-only by design**. Cosign says so before it proceeds:

```
This information will be used for signing this artifact and will be stored in
public transparency logs and cannot be removed later
```

That is not a warning about a preference. It means your personal email address becomes a permanent public record, attached to a throwaway lab artifact, and no one can delete it afterward. Transparency logs are immutable for the same reason your evidence vault is, which is a good property in the right place and a poor trade here.

Lab 4.4 does it the way it should be done. There, signing happens inside the pipeline, and Fulcio issues a certificate whose subject is the **repository, workflow and ref** rather than a human:

```
https://github.com/OWNER/REPO/.github/workflows/grc-gate.yml@refs/heads/main
```

Nothing personal enters the log, the identity is more useful for auditing because it names the process rather than whoever happened to be at a keyboard, and there is no long-lived credential involved either. Waiting for 4.4 costs you nothing and teaches the correct shape.

If you do want to see a signature today, use a throwaway account, or sign something you do not mind being publicly associated with forever. Not your lab evidence, and not with the address you use for work.

Record the vault in `cgep.env`

Lab 5.2 scopes CloudTrail data events to this vault, and needs its ARN:

```
vault=$(terraform output -raw vault_name) &&
  echo "export TF_VAR_evidence_vault_arn=arn:aws:s3:::$vault" >> ../../../cgep.env
source ../../../cgep.env
```

Do this after the apply, not after the plan. Outputs do not exist until the resources do. The `&&` guard is what stops a premature run appending Terraform's "No outputs found" warning into `cgep.env`, where sourcing the file then tries to execute it.

Verification

Every check below asserts a value, which the describe calls this section used to print could not. `aws s3api get-bucket-encryption` exits 0 whether the answer is `aws:kms` or `AES256`, and `get-bucket-versioning` prints nothing and still exits 0 on a bucket where versioning was never enabled. Object Lock in COMPLIANCE mode means nobody deletes the object, including you, including the account root, until the retention date passes. Reading a describe call that says so is not the same as watching the delete fail.

Pass the workspace, so it does not matter where you run it from:

```
bash scripts/verify.sh terraform
```

Every line names a control and either passes or fails with what it expected and what the account actually returned. The run ends in `VERIFIED` or `NOT VERIFIED` and sets the exit status accordingly.

```
#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 2.5. Every check asserts a value and exits non-zero if
# the account disagrees.
```

```
#
# This lab's controls are the ones most worth watching refuse. Object Lock in
# COMPLIANCE mode means nobody deletes the object, including you, including
# the account root, until the retention date passes. Reading a describe call
# that says so is not the same as watching the delete fail.
set -uo pipefail

# Point this at the workspace whose `terraform output` describes the resources
# you want checked, so it does not matter where you run it from. Guessing the
# working directory is how a verification script cheerfully checks the wrong
# account and passes.
WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0

pass() { printf '  PASS  %-8s %s\n' "$1" "$2"; }
fail() { printf '  FAIL  %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

check() { # check CONTROL LABEL EXPECTED ACTUAL
  if [[ "$3" == "$4" ]]; then
    pass "$1" "$2 is $4"
  else
    fail "$1" "$2: expected '$3', got '${4:-(nothing)}'"
  fi
}

refuse() { # refuse CONTROL LABEL COMMAND...
  local control=$1 label=$2 out rc
  shift 2
  out=$("$@" 2>&1)
  rc=$?
  if [[ $rc -ne 0 && "$out" == *AccessDenied* ]]; then
    pass "$control" "$label was refused"
  else
    fail "$control" "$label was NOT refused (exit $rc)"
  fi
}

VAULT=$(terraform output -raw vault_name)
MODE=$(terraform output -raw lock_mode)
KEY="runs/test-001/bundle.tar.gz"

# capture-evidence.sh sets VERSION_ID inside its own process, which never
# reaches yours, so derive it here rather than assuming it is set.
VERSION_ID=$(aws s3api list-object-versions --bucket "$VAULT" \
  --prefix "runs/test-001/" --query 'Versions[0].VersionId' --output text 2>/dev/null)

echo "=== configuration ==="

check SI-7 "object lock" "Enabled" \
  "$(aws s3api get-object-lock-configuration --bucket "$VAULT" \
       --query 'ObjectLockConfiguration.ObjectLockEnabled' --output text 2>/dev/null)"

check SC-28 "vault encryption" "aws:kms" \
  "$(aws s3api get-bucket-encryption --bucket "$VAULT" \
```

```
       --query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.SSEAlgor
ithm' \
       --output text 2>/dev/null)"

# The mode is the whole lesson. GOVERNANCE can be bypassed by anyone holding
# s3:BypassGovernanceRetention; COMPLIANCE cannot be bypassed by anyone.
check AU-9 "retention mode" "$MODE" \
  "$(aws s3api get-object-retention --bucket "$VAULT" --key "$KEY" \
       --query 'Retention.Mode' --output text 2>/dev/null)"

if [[ -z "$VERSION_ID" || "$VERSION_ID" == "None" ]]; then
  fail "AU-9" "no object version found under runs/test-001/. Run capture-evidence.sh first."
else
  pass "AU-9" "object version present: $VERSION_ID"
fi

echo "=== enforcement ==="

refuse SC-8 "plain HTTP" \
  aws s3api list-objects-v2 --bucket "$VAULT" \
  --endpoint-url "http://s3.us-east-1.amazonaws.com"

# The versioned delete is the real test. Deleting without a version id only
# writes a delete marker, which succeeds and proves nothing: the object is
# still there underneath. Naming the version is what Object Lock refuses.
if [[ -n "$VERSION_ID" && "$VERSION_ID" != "None" ]]; then
  refuse AU-9 "versioned delete under $MODE retention" \
    aws s3api delete-object --bucket "$VAULT" --key "$KEY" --version-id "$VERSION_ID"
fi

# The misleading success, asserted rather than described. A delete with no
# version id writes a delete marker: the call succeeds, the object disappears
# from listings, and nothing was destroyed. It is the most misread result in
# this lab, so the script both proves it and then puts the vault back.
if aws s3api delete-object --bucket "$VAULT" --key "$KEY" >/dev/null 2>&1; then
  pass "AU-9" "unversioned delete succeeded, having written only a delete marker"
  MARKER=$(aws s3api list-object-versions --bucket "$VAULT" --prefix "$KEY" \
    --query 'DeleteMarkers[0].VersionId' --output text 2>/dev/null)
  if [[ -n "$MARKER" && "$MARKER" != "None" ]]; then
    # Delete markers carry no retention of their own, so this is permitted
    # even under COMPLIANCE mode, and it leaves the object visible again.
    if aws s3api delete-object --bucket "$VAULT" --key "$KEY" \
         --version-id "$MARKER" >/dev/null 2>&1; then
      pass "AU-9" "delete marker removed, the object is listed again"
    else
      fail "AU-9" "delete marker $MARKER could not be removed. Remove it by hand."
    fi
  fi
else
  fail "AU-9" "unversioned delete was refused, which is not how Object Lock behaves"
fi

echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: every control asserted above holds in the account."
```

```
else
  echo "NOT VERIFIED: at least one control does not hold. Do not capture evidence yet." >&2
fi
exit $FAILED
```

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-2-5"
mkdir -p "$EVIDENCE"

# The receipt: re-run the capture and keep its output this time.
scripts/capture-evidence.sh \
  --workspace ../lab-2-3 \
  --run-id    test-001 \
  --vault     "$VAULT" | tee "$EVIDENCE/receipt.json"

# The lock test: both refusals, the misleading success, and the cleanup.
bash scripts/verify.sh terraform 2>&1 | tee "$EVIDENCE/lock-test.txt"
```

Portfolio submission checklist

- [ ] `terraform/primitives/evidence-vault/` deploys the vault as shown.
- [ ] `scripts/capture-evidence.sh` committed and executable.
- [ ] At least one bundle uploaded, VersionId recorded in `evidence/lab-2-5/receipt.json`.
- [ ] `evidence/lab-2-5/lock-test.txt`, a transcript of the failed versioned delete.
- [ ] README states your `lock_mode` and `retention_days` **and defends them**.
- [ ] README states whether you capture state, and why. This is new here, and it is the question a data-protection reviewer asks first.

Troubleshooting

- **`Saved plan is stale`** when you apply. Something changed the state between your `plan -out=tfplan` and your `apply`, and a saved plan is a promise about one specific state. Often it is innocuous: a `terraform output` or `refresh` recording a data source counts. Plan again, read the new plan, apply that. Nothing is lost and nothing is broken; Terraform cannot distinguish a harmless change from a dangerous one, so it refuses instead of guessing.
- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-`

`credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.

- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`InvalidBucketState: Object Lock configuration cannot be enabled on existing buckets`.** Object Lock is creation-time only. There is no upgrade path. Destroy and recreate.
- **`COMPLIANCE` mode with too long a retention.** You cannot shorten or remove it. The objects sit until they expire, billing the whole time. The `acknowledge_compliance_mode` variable exists to make you type something before this can happen.
- **`--include-state` refused.** Working as designed. Read Step 3 and decide deliberately.
- **Clock drift.** `RetainUntilDate` is wall-clock UTC. If your laptop or runner clock is skewed, retention math will surprise you. Trust the server's date.
- **Cosign keyless from a laptop** needs a browser for the OIDC flow. In GitHub Actions it is automatic with `permissions: id-token: write`.
- **`delete-objects` fails with `MalformedXML`.** You passed an empty `Objects` list. Guard on length; see Cleanup.

Cleanup

`GOVERNANCE` allows bypass. `COMPLIANCE` does not, and if you chose it, the bucket stays until every retention expires. For a lab, that is the wrong choice; for production evidence, it is the point.

```
VAULT=$(terraform output -raw vault_name)

aws s3api list-object-versions --bucket "$VAULT" --output json \
  | jq '{Objects: [(.Versions // []), (.DeleteMarkers // [])] | flatten | .[] | {Key, VersionId}} ' \
  | jq -s '{Objects: map(.Objects)}' > /tmp/del.json

if [ "$(jq '.Objects | length' /tmp/del.json)" -gt 0 ]; then
  aws s3api delete-objects --bucket "$VAULT" --delete file:///tmp/del.json \
    --bypass-governance-retention
fi

terraform destroy -auto-approve
```

The length guard is there instead of a trailing `|| true`, which would swallow the empty-list error and every other error along with it. A cleanup script that cannot fail is a cleanup script that cannot tell you it failed.

The vault CMK enters its deletion waiting period and bills $1/month throughout.

How this feeds the capstone

This vault **is** the capstone's evidence vault.

- **Ch 4.3 and 4.4**, every PR runs the pipeline, which signs the bundle and uploads it here with this exact pattern.
- **Ch 5.2**, you turn on CloudTrail S3 data events for this bucket specifically. It is the one bucket where "who read the evidence" is itself an audit question.
- **Ch 6.1**, your OSCAL component's `links[rel=evidence].href` resolves to an object in here, by VersionId.
- **The grader** downloads from here and runs `verify-evidence.sh`. Build it once, well.

Revision history

These notes

- Vault encryption moved from SSE-S3 to a dedicated customer-managed KMS key, separate from the workload key, adding SC-12 and SC-13.
- Added a bucket policy denying `aws:SecureTransport = false`, adding SC-8.
- `capture-evidence.sh` no longer captures raw Terraform state by default. It bundles the plan and the module attestation instead, and refuses `--include-state` against a COMPLIANCE-mode vault without an explicit acknowledgment.
- Added an `acknowledge_compliance_mode` guard so COMPLIANCE retention beyond 7 days must be deliberate.
- The receipt records whether state was included, and the VersionId is read with `--query` rather than parsed out of JSON by hand.
- Cleanup guards on object-list length instead of relying on `|| true`.

The official labs

Initial release: AES256 vault, `terraform state pull` captured into every bundle by default, no TLS-deny policy.